Surname, Initials:	Student ID:	Date:

CSI243 Functional Programming
Lab 2 Recursion and Lists

Objectives

1. **Recursion**
2. **Applying recursion to basic lists**

Background / Scenario

Instead of using for or while loops, **recursion** is used for looping in Haskell. A recursive function **defines the function in terms of itself**. The function invokes itself only when a condition is met, as with an if/else/then expression. As in any form of loops, there has to be at least one case that terminates the recursion, and a recursive case that will cause the function to call itself.

So, to represent a recursive function, there should be two parts:

- The **base case** – defines the function for the first input values, terminates the recursion.
- The **recursive step** – defines the function for other input values in terms of itself, creating a loop

Recursion can be implemented using if then statements, or using a finite number of guards, where the first guard evaluates to true.

Haskell has many recursive functions, especially concerning lists.

For each of the questions below, there should be three parts:

i. Specify the signature of the function, that is, the arguments and the return type the function

ii. Give the base and the recursive definition of the function using guards

iii. Dry run the function

iv. Then implement and test

Task 1 (this is from the previous test)

Specify and implement a recursive function `sumNumbers` that takes an integer variable `n` and computes the sum of all positive whole numbers from 0 up to, but **not including** n.

For example: `sumNumbers 10` will return 1+2+3+4+5+6+7+8+9 = 45

Task 2

Specify and implement a recursive function `powerNum` that takes two integers `n` and `y` and raises `n` to power `y`.

For example: `powerNum 2 7` should return 2^7 = 128.

Remember that any number raised to power 0 returns 1, and any number raised to power 1 returns that number. That is, powerNum 10 0 should return 1 and powerNum 10 1 should return 10.

Tasks 3

Create a module `ListFunctions` containing the recursive list functions below. For each function, give the specification, dry run and implement.

a) `listLength` gives the number of elements in a list
Remember that the length of an empty list is 0.

b) `concatLists` that takes two lists of the same type, concatenates them, and produce another list of the same type.
As an example, `concatLists [1,2,3] [4,5,6]` should give `[1,2,3,4,5,6]`

c) `elementOccurs` gives the number of occurrences of a given element in a list. As an example, `elementOccurs [1,2,3,1,3,1] 1` should return 3.

d) isElementIn that checks if an element occurs in a list. As an example, `isElementIn [7,10,6,0] 6` should return `True`, on the other hand, `isElementIn [7,10,6,0] 5` should return `False`.